Smallest Polyiamond Containing All Free n-Iamonds

Peter Exley
pexley-math@github
April 2026

Abstract. We determine $a(n)$, the minimum number of cells in a connected polyiamond that contains every free n-iamond as a cell-subset (an isomorphic embedding into the cell set of the container) under rigid motions of the triangular lattice (D_6 orientations plus parity-preserving lattice translations). This is the triangular-lattice analogue of T. R. Dawson's 1942 Minimum Common Superform problem for pentominoes, and the companion to OEIS A327094 (square lattice). We determine

```
a(1..14) = 1, 2, 3, 5, 7, 10, 13, 16, 19, 24, 27, 32, 37, 42
```

by exact SAT search with lazy connectivity cuts, proving each value via matching SAT/UNSAT certificates. Every reported solution is verified by two independent geometric verifiers – one re-implementing the full D_6 action in offset coordinates, one using a separate library primitive – both of which restrict to parity-preserving lattice translations (the rigid motions of the triangular tiling). A single architectural choice – a tight rectangular grid of $\max(4, \lfloor (n+2)/3 \rfloor)$ rows by n columns – yields more than two orders of magnitude runtime speed-up at $n = 11$ over a naive $n \times n$ grid baseline on our hardware (AMD Ryzen 5 5600). Based on computed ratios $a(n)/n^2$ monotonically decreasing from 0.240 to 0.214 over $n = 10..14$, we conjecture

$$0 < \liminf_{n\to\infty} a(n)/n^2 \leq \limsup_{n\to\infty} a(n)/n^2 \leq 0.24$$

without asserting that the full limit exists. The informal upper bound $a(n) \leq \lceil (n^2 + 2n)/5 \rceil$ holds on all computed terms, tight at $n \leq 8$ and $n = 10$, and loose by 1–3 cells for $n \in \{9, 11, 12, 13, 14\}$. We document several failed closed-form candidates and propose concrete directions (residue-class piecewise formulas, recurrences, isoperimetric lower bounds) for future formula searches.

Keywords: polyiamond, triangular lattice, minimum common superform, universal container, SAT solver, connectivity cuts

MSC: 05B50, 52C20, 68R05

---

1. Introduction

The *Minimum Common Superform (MCS)* problem asks: given a set S of shapes, what is the smallest single shape that contains every element of S as a sub-polyiamond (i.e. as a cell-subset, up to the symmetries of the ambient lattice)? T. R. Dawson raised this question for the twelve pentominoes in *Fairy Chess Review* Vol. 5, No. 4 (1942); the set of pentominoes has two distinct 9-cell MCSs.

A natural generalisation is the *universal container* problem: fix a lattice and a size parameter n, and take S to be the set of all free n-cell shapes on that lattice. Then $a(n)$ is the number of cells in the smallest connected shape that contains every member of S as a cell-subset.

Context and prior work. For the square lattice, $a(n)$ is OEIS A327094, proved through $n = 9$ in 2019 and unchanged since; no closed-form formula is known. The input to our problem – the number of free n-iamonds – is OEIS A000577, known through $n = 14$ (26,166 pieces). No triangular-lattice version of the universal container problem has been previously reported in OEIS, arXiv, Google Scholar, GitHub, or general web sources; a comprehensive prior-art search across these archives (documented in the project repository) returned no matches. No systematic study of universal containers for free n-hexes on the hexagonal lattice appears in the literature either, to our knowledge. SAT-based enumeration of polyforms has been used in tools such as `heesch-sat` for isohedral-tiling problems; we are not aware of prior application to universal containers. To our knowledge, this is the first systematic study of universal containers on the triangular lattice.

Main result.

> **Theorem 1.** Let $a(n)$ be the minimum number of cells in a connected polyiamond such that every free n-iamond embeds into the container under a rigid motion of the triangular lattice (a D_6 orientation plus a parity-preserving lattice translation). Then
>
> `a(1..14) = 1, 2, 3, 5, 7, 10, 13, 16, 19, 24, 27, 32, 37, 42.`

Each value is established by SAT search (matching SAT at $k = a(n)$ and UNSAT at $k = a(n) - 1$) on a tight grid, and verified by brute-force enumeration with a rigid-motion containment check: every free n-iamond is tested against every D_6 orientation, under every parity-preserving lattice translation, inside the reported container. Every reported container is additionally checked by two independent geometric verifiers (`verify_independent.py`, `verify_geometric.py`) that share no code paths with the solver, both of which confirm all 14 values (Section 7). A grid-widening grid-sufficiency verification protocol for detecting tight-grid insufficiency is documented in Section 6; this paper's primary independent validation is the pair of geometric verifiers.

We study the triangular lattice, denoted T, whose cells are unit equilateral triangles joined along edges. Each cell has only three edge-neighbours (vs four on the square lattice, six on the hexagonal), which makes the constituent shapes "thin" and gives tighter packing than the square lattice. Empirically, for $n \geq 4$, the triangle values are smaller than the square values (cf. Table 3). We have not yet computed universal container sizes on the hexagonal lattice; we conjecture that the square-lattice values in turn lie below the eventual hexagonal-lattice values, but make no formal claim.

Contributions.

1. We establish $a(n)$ exactly for $n = 1..14$ (Theorem 1) by SAT search with brute-force verification of each reported container. Every reported container is additionally checked by two independent geometric verifiers that re-implement the D_6 action and the rigid-motion containment check with disjoint code paths.

2. We introduce a tight rectangular grid heuristic $R(n) = \max(4, \lfloor (n+2)/3 \rfloor)$ which, combined with a standard binary-search-plus-linear-descent top-down scheme, gives two orders of magnitude speed-up over the natural $n \times n$ grid. The heuristic is formula-driven and uses no hard-coded $a(n)$ values.

3. We specify a grid-widening grid-sufficiency verification protocol (full binary-plus-linear search on the $(R(n)+1) \times n$ grid, with independent SAT and UNSAT certificates) for detecting tight-grid-insufficiency failures when extending the sequence to $n \geq 15$ (Section 6). The protocol is complementary to the two independent geometric verifiers (Section 7), which are the primary validation for the reported $a(n)$ values.
4. We report a failed search for a closed-form expression across five structurally different candidate families (pure quadratics, triangular-grid isoperimetric, Fibonacci-style recurrences, cross-grid shifts from A327094, polyiamond-hole style offsets). None matches all 14 proved terms. The closest match `ceil((n^2 + 2n)/5)` is tight for $n \leq 8$ and $n = 10$, and loose by 1–3 cells for $n \in \{9, 11, 12, 13, 14\}$, documented as an informal upper bound only.
5. We record the asymptotic observation $a(n)/n^2$ monotonically decreasing over $n = 10..14$ (0.240, 0.223, 0.222, 0.219, 0.214) and propose six concrete directions for future formula search (Section 9.2).

---

2. Definitions and Lattice Conventions

Notation summary. T = triangular lattice (unit equilateral triangles joined along edges). (r, c) = offset coordinates. D_6 = dihedral group of order 12. R_{60}, F = generators of D_6. $\mathrm{orient}(P)$ = set of D_6 images of polyiamond P. $a(n)$ = minimum cells in a connected polyiamond containing every free n-iamond. $R(n) = \max(4, \lfloor (n+2)/3 \rfloor)$ = tight-grid row count. $A000577(n)$ = number of free n-iamonds. A rigid motion = D_6 element + parity-preserving translation ($(dr + dc)$ even).

2.1 Coordinates and orientation

We describe T via *offset coordinates*: each cell is labelled (r, c) with $r, c \in \mathbb{Z}$. We refer to r as the "row" and c as the "column" for algorithmic convenience, even though the cells themselves are equilateral triangles and not axis-aligned squares. The *orientation* (up-pointing apex-at-top or down-pointing apex-at-bottom) of cell (r, c) is determined by parity. Following the standard triangular-grid convention (Lunnon 1972; Polyform Puzzler; BorisTheBrave):

```
(r, c) is down-pointing (apex at bottom) iff (r + c) is even;
(r, c) is up-pointing   (apex at top)    iff (r + c) is odd.
```

Here "x mod 2" denotes the remainder of x on division by 2, so x mod 2 is in $\{0, 1\}$.

A down-pointing cell (r, c) (i.e. $(r + c)$ even) has edge-neighbours $(r, c-1)$, $(r, c+1)$, $(r-1, c)$; an up-pointing cell (i.e. $(r + c)$ odd) has neighbours $(r, c-1)$, $(r, c+1)$, $(r+1, c)$.

A small patch of the lattice illustrates the coordinate system. Down-cells (D, triangle apex-at-bottom) have $(r + c)$ even; up-cells (U, triangle apex-at-top) have $(r + c)$ odd. Horizontal neighbours always differ in parity; a down-cell's third neighbour lies above it, an up-cell's third neighbour lies below it:

```
    row r=0:    D  U  D  U  D  U        (cells (0,0), (0,1), (0,2), (0,3), (0,4),
(0,5))
    row r=1:    U  D  U  D  U  D        (cells (1,0), (1,1), (1,2), (1,3), (1,4),
(1,5))
    row r=2:    D  U  D  U  D  U        (cells (2,0), (2,1), (2,2), (2,3), (2,4),
(2,5))
```

2.2 Translations and the symmetry group D_6

The *translation subgroup* of the triangular lattice – the set of integer shifts (dr, dc) that map the cell set to itself while preserving each cell's orientation – is generated by $(0, 2)$ and $(1, 1)$, equivalently the set of offsets (dr, dc) with $(dr + dc)$ even. These are the only translations that are rigid Euclidean motions of the tiling; shifts with $(dr + dc)$ odd are NOT translations of the lattice, as they flip every cell's orientation per-cell and do not correspond to any rigid motion of the triangular tiling. The symmetry group acting on (unlabelled) polyiamonds is the dihedral group D_6 of order 12: six rotations by multiples of 60 degrees combined with a reflection. Section 2.5′s placement generator (Algorithm 1) correspondingly pairs every orientation in $\text{orient}(P)$ with every parity-preserving lattice translation, and SKIPS parity-flipping shifts.

We use the following explicit generators for D_6 on (r, c) coordinates:

- **Rotation by 60 degrees clockwise:**

```
R_60(r, c) = ( floor((r + c + 1) / 2),
               floor((c - 3*r - ((r+c) mod 2) - 2) / 2) ).
```

- **Reflection (horizontal across a lattice line):**

```
F(r, c) = (-r - 1, c).
```

The 12 elements of D_6 are generated by applying R_{60} zero to five times, optionally preceded by F.

Definition as lattice symmetries. The formulas above are defined axiomatically on $\mathbb{Z}^2$, and we verify in Appendix A that they generate a D_6 subgroup of the automorphism group of the triangular-lattice edge-adjacency graph. Specifically: $R_{60}^6 = \text{identity}$, $F^2 = \text{identity}$, $FR_{60}F = R_{60}^{-1}$ (the dihedral relation), and each of R_{60} and F is a graph automorphism preserving the edge-neighbour relation of Section 2.1. We also verified orbit counts for each free n-iamond with $n \leq 14$ are consistent with A000577 (summed over orbit representatives). Note that R_{60} is *not* a rigid Euclidean rotation when cells are placed at centroids of unit equilateral triangles; it is an abstract combinatorial symmetry of the integer lattice that coincides with a D_6 subgroup of the graph automorphism group. See Appendix A.

Parity preservation. The translation by (dr, dc) with $(dr + dc)$ even preserves the up/down orientation of every cell (since $(r + c) \bmod 2$ and $((r + dr) + (c + dc)) \bmod 2$ differ by $(dr + dc) \bmod 2 = 0$). Conversely, $(dr + dc)$ odd flips every cell's orientation, turning a polyiamond into its "parity-swapped" version.

The normalisation step in Algorithm 1 translates every cell by $(-r_{\min}, -\text{shift}_c)$ where

```
shift_c = r_min + 2 * floor((c_min - r_min) / 2).
```

Lemma 2.1 (parity-preserving normalisation). The translation $(dr, dc) = (-r_{\min}, -\text{shift}_c)$ satisfies $dr + dc$ even, so normalisation preserves every cell's up/down orientation.

Proof. By definition $\text{shift}_c = r_{\min} + 2 \cdot \lfloor (c_{\min} - r_{\min})/2 \rfloor$, so $dr + dc = -r_{\min} - \text{shift}_c = -2r_{\min} - 2 \cdot \lfloor (c_{\min} - r_{\min})/2 \rfloor$, which is an even integer. Thus $(dr + dc) \bmod 2 = 0$ and, by the parity argument at the start of Section 2.2, the orientation of every cell is preserved. After the translation the new minimum column is $c_{\min} - \text{shift}_c = (c_{\min} - r_{\min}) - 2 \cdot \lfloor (c_{\min} - r_{\min})/2 \rfloor = (c_{\min} - r_{\min}) \bmod 2$, which is 0 when $(c_{\min} - r_{\min})$ is even and 1 when odd. □

Worked normalisation examples.

- $r_{\min} = 0$, $c_{\min} = 1$: $\text{shift}_c = 0 + 2 \cdot 0 = 0$; new min col $= 1 - 0 = 1$ (odd case).
- $r_{\min} = 0$, $c_{\min} = 2$: $\text{shift}_c = 0 + 2 \cdot 1 = 2$; new min col $= 2 - 2 = 0$ (even case).

- $r_{\min} = 1$, $c_{\min} = 2$: $\text{shift}_c = 1 + 2 \cdot 0 = 1$; new min col $= 2 - 1 = 1$ (odd case).

Worked R_{60} example (2-iamond). Consider the canonical 2-iamond with cells $\{(0,0),(0,1)\}$. Under the convention of Section 2.1, cell $(0,0)$ has $(0+0) = 0$ even so it is *down-pointing* (apex at bottom); cell $(0,1)$ has $(0+1) = 1$ odd so it is *up-pointing* (apex at top). Applying R_{60}:

- $R_{60}(0,0)$: $t = 0 \bmod 2 = 0$, so $r' = \lfloor (0+0+1)/2 \rfloor = 0$, $c' = \lfloor (0-0-0-2)/2 \rfloor = \lfloor -2/2 \rfloor = -1$. Result: $(0,-1)$.
- $R_{60}(0,1)$: $t = 1 \bmod 2 = 1$, so $r' = \lfloor (0+1+1)/2 \rfloor = 1$, $c' = \lfloor (1-0-1-2)/2 \rfloor = \lfloor -2/2 \rfloor = -1$. Result: $(1,-1)$.

Rotated cell set: $\{(0,-1),(1,-1)\}$. This is geometrically the original 2-iamond rotated 60 degrees clockwise: the horizontal adjacent pair becomes a vertically adjacent pair. Normalising: $r_{\min} = 0$, $c_{\min} = -1$, so $\text{shift}_c = 0 + 2 \cdot \lfloor (-1-0)/2 \rfloor = 0 + 2 \cdot (-1) = -2$. Translating by $(0,2)$ gives $\{(0,1),(1,1)\}$, with new min column $(c_{\min} - \text{shift}_c) = -1 - (-2) = 1$. (The normalised form has min_c $= 1$ because $(c_{\min} - r_{\min}) = -1$ is odd.) Up to a parity-preserving translation in the lattice, this is the same 2-iamond – the 2-iamond is invariant under D_6.

Orbit parity coverage. Both R_{60} and F are parity-flipping maps on the cell lattice.

- **R_{60} flips parity.** For any cell (r,c) with parity $t = (r+c) \bmod 2$, applying R_{60} gives (r',c') with $r' = \lfloor (r+c+1)/2 \rfloor$ and $c' = \lfloor (c-3r-t-2)/2 \rfloor$. A direct case split on $t \in \{0,1\}$ (carried out in Appendix A.4) yields $(r'+c') \bmod 2 = 1 - t$. Thus R_{60} maps up-cells to down-cells and vice versa.

- **F flips parity.** For $F(r,c) = (-r-1, c)$, compute $(-r-1+c) \bmod 2 = (c-r-1) \bmod 2 = (r+c+1) \bmod 2$ (since $(c-r)$ and $(r+c)$ have the same parity), which is $1-t$. Thus F also flips parity.

Since both generators of D_6 flip parity, any element of D_6 is parity-preserving if and only if it is a product of an even number of generators (e.g., R_{60}^2, $FR_{60}FR_{60}$); it is parity-flipping for an odd number (e.g., R_{60}, F, R_{60}^3). The group D_6 contains 6 parity-preserving elements and 6 parity-flipping elements. Therefore, the orbit of any cell under D_6 intersects both parity sublattices, and $\text{orient}(P)$ contains representatives in both sublattices.

2.3 Free polyiamonds and orient(P)

A *polyiamond* is a finite connected set of cells; a *free polyiamond* is an equivalence class under the action of D_6 together with translations of the lattice. For a polyiamond P, we write $\text{orient}(P)$ for the set of up to 12 distinct orientations obtained by applying all 12 elements of D_6 to P and normalising each.

Normalisation. Given a cell set S, let $r_{\min} = \min\{r : (r,c) \in S\}$, $c_{\min} = \min\{c : (r,c) \in S\}$. We translate S by $(-r_{\min}, -dc)$ where dc is chosen to make the translated $c_{\min}$ equal to 0 if $(c_{\min} - r_{\min})$ is even, or 1 if $(c_{\min} - r_{\min})$ is odd. This preserves lattice parity (i.e. leaves orientation correct) and gives a canonical representative.

2.4 The "fits in" relation

A free polyiamond P *fits in* a container $C \subseteq T$ under a rigid motion if there exists an orientation $o \in \text{orient}(P)$ and an offset $(dr, dc) \in \mathbb{Z}^2$ **with $(dr + dc)$ even** such that

```
{ (r + dr, c + dc) : (r, c) in o }  is a subset of C.
```

The parity constraint $(dr + dc)$ even is essential: it restricts to rigid motions of the triangular tiling. Allowing odd-parity shifts would admit placements that are not congruent to P – the placed cells would not have the same orientation pattern as P or any D_6 image of P (see Section 2.2 and Appendix A).

Why the parity constraint matters. A D_6 element g applied to P produces an orientation $g(P)$ with a specific per-cell orientation pattern (sequence of up/down labels). A parity-preserving translation ($(dr + dc)$ even) preserves this pattern. A shift with $(dr + dc)$ odd flips every cell's orientation, producing a cell set that looks like the pattern-inverse of $g(P)$. This inverted pattern is NOT congruent to P under any element of D_6 (the D_6 orbit of P does not include pattern-inverses in general; see the worked counter-example in Appendix A). Iterating over odd-parity offsets therefore admits spurious "placements" that are not rigid-motion embeddings of P.

Algorithm 1 (Section 2.5) correctly enforces $(dr + dc)$ even, and the independent geometric verifier `verify_independent.py` likewise restricts to parity-preserving lattice translations.

2.5 Placement generation (Algorithm 1)

Algorithm 1: Placement Generation.
Input: free n-iamond P (canonical cell set), grid $G = \{0, ..., R-1\} \times \{0, ..., C-1\}$.
Output: set of distinct placements of P inside G.

```
orientations := empty set
for k in 0..5:
    for refl in {no, yes}:
        shape := P
        if refl: shape := { F(r,c) for (r,c) in shape }
        apply R_60 k times to shape
        add normalise(shape) to orientations
placements := empty set
for each o in orientations:
    (r_max, c_max) := pointwise max of o
    (r_min, c_min) := pointwise min of o
    for dr in range(-r_min, R - r_max):
        for dc in range(-c_min, C - c_max):
            if (dr + dc) mod 2 != 0:
                continue    // skip parity-flipping shifts: not rigid motions
            p := tuple(sorted((r+dr, c+dc) for (r,c) in o))
            add p to placements
return placements
```

Deduplication is accomplished implicitly by using `placements` as a set keyed on the lex-sorted tuple. Self-symmetric pieces (where the D_6 action has non-trivial stabiliser) produce fewer than 12 orientations; the algorithm handles this automatically.

---

3. The Decision Problem and SAT Encoding

For each n and each candidate size k, we decide whether a k-cell connected container exists. Fix a grid $G = \{0, ..., R-1\} \times \{0, ..., C-1\}$. Enumerate the free n-iamonds $P_1, ..., P_m$ (where $m = A000577(n)$); then for each P_i, enumerate its valid placements inside G via Algorithm 1.

Generation of the free n-iamonds. We enumerate the free n-iamonds by standard adjacency-based recursive expansion. Concretely, we start with $S_1 = \{\{(0, 0)\}\}$ (the unique

1-iamond). For each k from 2 to n, we build S_k as follows: for each shape Q in S_{k-1}, for each cell (r, c) in Q, and for each edge-adjacent cell (r', c') not already in Q, form $Q' = Q \cup \{(r', c')\}$. Compute the canonical form of Q' as the lexicographic minimum of sorted-tuple representations over the full D_6 orbit (each orbit representative normalised via Lemma 2.1), and insert the canonical form into S_k (a set), which deduplicates. The resulting $|S_k|$ is cross-checked against $A000577(k)$ at solver startup; any discrepancy aborts the run. A Cython-accelerated implementation (polyform_enum) is used when available; otherwise a pure-Python fallback produces identical results.

Boolean variables. For each cell (r, c) in G introduce $x_{r,c} \in \{0, 1\}$ with $x_{r,c} = 1$ iff (r, c) is in C. For each placement p of piece i, introduce an auxiliary $y_{i,p} \in \{0, 1\}$.

Constraints.

- **Coverage.** For each piece i, $\sum_p y_{i,p} \geq 1$ (some placement is active).
- **Activation.** For each placement p of piece i and each cell (r, c) in p: $y_{i,p} \to x_{r,c}$.
- **Cardinality.** Exactly k cells occupied: $\sum_{r,c} x_{r,c} = k$, enforced via the totalizer encoding – a standard CNF encoding that uses a sorting network of auxiliary variables to represent arbitrary cardinality constraints in $O(N \log N)$ clauses (Bailleux and Boufkhad, "Efficient CNF Encoding of Boolean Cardinality Constraints", CP 2003; implemented by PySAT's CardEnc.equals with EncType.totalizer).
- **Minimum degree (applied only for $k \geq 2$).** Each occupied cell has at least one occupied neighbour: $x_{r,c} \to \bigvee_{\text{nb} \in \text{nbrs}(r,c)} x_{\text{nb}}$. This is a necessary consequence of connectivity for $k \geq 2$ – any cell in a connected set of size ≥ 2 has at least one neighbour – and is included explicitly to let the solver prune isolated cells early. It excludes no valid container for $k \geq 2$. For $k = 1$ the constraint is omitted (a single cell trivially satisfies $a(1) = 1$ with no neighbours required).
- **Shape constraints (heuristic, for $n \geq 6$).** (a) Contiguous columns per row, (b) at least one row with all C columns occupied. See Section 3.1. Our use of these is carefully scoped: see Observation 3.1 below.
- **Connectivity.** Enforced lazily via iterative connectivity cuts. We fix a total order on cells by $(r, c) < (r', c')$ iff $r < r'$ or ($r = r'$ and $c < c'$); the notation $\min(\text{component})$ below denotes the least cell in this order. After each SAT call we extract the model assignment into the cell set $M := \{(r, c) \in G : x_{r,c} = 1\}$ and compute connected components of M under the edge-neighbour relation on T via a BFS restricted to M. The procedure is:

```
components := connected components of M under the edge-neighbour
              relation on T (BFS restricted to M)
sort components by (-len(component), min(component))
    # descending size; lex-smallest cell as tie-breaker
largest := components[0]
for comp in components[1:]:
    add_clause( OR over (r,c) in comp of NOT x_{r,c} )
```

This deterministic rule selects a unique "largest" component even under ties and forbids every other component in the next round. Iterate until a single component remains.

Remark (complexity of connectivity cuts). Each iteration in which the model has $t > 1$ components adds $t - 1$ blocking clauses. Each blocking clause has at most k literals. The iteration strictly increases the number of cuts without decreasing k, and since there are only finitely many k-subsets of G the procedure must terminate; empirically termination occurs

after $O(k)$ or fewer iterations in our runs, giving $O(k)$ added clauses of at most k literals each, i.e. $O(k^2)$ extra literals overall, which is polynomial in $R \cdot C$.

Connectivity-cut worked example. Suppose at iteration t the model has cells $\{(0,0),(0,1),(2,0),(2,1)\}$ with $k=4$. Running BFS over the edge-neighbour relation yields two components $C_1 = \{(0,0),(0,1)\}$ and $C_2 = \{(2,0),(2,1)\}$, both of size 2. Tie-breaking by lex-smallest cell (row, then column): the smallest cell of C_1 is $(0,0)$, the smallest of C_2 is $(2,0)$, and $(0,0) < (2,0)$, so C_1 is the largest. We add the cut `(NOT x_{2,0}) OR (NOT x_{2,1})` forbidding the C_2 cell-set. The next iteration re-solves and must place cells somewhere other than both $(2,0)$ and $(2,1)$ simultaneously.

Under this encoding, SAT certifies an upper bound; UNSAT at $k-1$ certifies the lower bound. The pair proves $a(n) = k$ exactly.

Example ($n = 4$, worked). There are three free tetraiamonds: the triangle (4 cells forming an equilateral triangle of side 2), the straight I-tetraiamond (4 cells in a line), and the C-tetraiamond (a 3-cell row with one cell stacked on the end). Take $R = 4$, $C = 4$. With the shape constraint off ($n < 6$ so it is not applied), at $k = 4$ the solver reports UNSAT: no 4-cell connected set contains all three pieces because the I-tetraiamond requires 4 cells in a single row while the triangle-tetraiamond requires a height-2 arrangement, and forcing both into 4 cells leaves no free cells to place the C-tetraiamond. At $k = 5$ the solver finds a satisfying witness: the 5-cell "rhombus with a tab" (cells $(0,0),(0,1),(0,2),(0,3),(1,1)$) contains all three pieces. Hence $a(4) = 5$.

Example ($n = 2$, smallest non-trivial). There is one free 2-iamond (the rhombus formed by an adjacent up and down triangle). Take $R = 2$, $C = 2$, so we have 4 cell variables $x_{0,0}, x_{0,1}, x_{1,0}, x_{1,1}$. After generating orientations via Algorithm 1, the piece has two distinct placements: $\{(0,0),(0,1)\}$ and $\{(0,1),(1,1)\}$. Coverage: $y_1 \vee y_2$. Activation: $y_1 \to (x_{0,0} \wedge x_{0,1})$, $y_2 \to (x_{0,1} \wedge x_{1,1})$. Cardinality: exactly 2 cells occupied. The solver immediately finds $x_{0,0} = x_{0,1} = 1$, so $a(2) = 2$.

3.1 Shape Constraint Encoding (heuristic, $n \geq 6$)

Let $x_{r,c}$ be the cell variable at row r, column c. For each row r and each triple $a < b < c$ with $a, c \in \{0, ..., C-1\}$ and $b \in (a, c)$:

```
x_{r,a} AND x_{r,c} -> x_{r,b}.
```

In CNF, each such contiguity constraint is the single clause

```
(NOT x_{r,a}) OR (NOT x_{r,c}) OR x_{r,b}.
```

For the "one full row" constraint, introduce auxiliary f_r for each row with

```
f_r <-> AND over c in {0, ..., C-1} of x_{r,c},
```

encoded by $C + 1$ clauses per row, plus the global disjunction

```
OR over r of f_r.
```

Worked example ($n = 6$, shape constraints). On grid $R = 4$, $C = 6$, row $r = 1$ generates contiguity clauses for every triple (a, b, c_{end}) with $a < b < c_{\text{end}}$. For example, $(a, c_{\text{end}}) = (0, 5)$ induces four clauses:

```
(NOT x_{1,0}) OR (NOT x_{1,5}) OR x_{1,1}
(NOT x_{1,0}) OR (NOT x_{1,5}) OR x_{1,2}
```

```
(NOT x_{1,0}) OR (NOT x_{1,5}) OR x_{1,3}
(NOT x_{1,0}) OR (NOT x_{1,5}) OR x_{1,4}
```

All triples over columns 0..5 together give a total of $C(C-1)(C-2)/6 = 20$ contiguity clauses per row. The full-row auxiliary for $r = 2$ uses $f_2 \leftrightarrow (x_{2,0} \wedge ... \wedge x_{2,5})$, adding $C + 1 = 7$ clauses:

```
(NOT f_2) OR x_{2,c}       for c = 0, 1, ..., 5
f_2 OR (NOT x_{2,0}) OR ... OR (NOT x_{2,5})
```

The global disjunction `f_0 OR f_1 OR f_2 OR f_3` forces at least one row to be completely filled.

Observation 3.1 (shape constraints are unsafe for $n \geq 11$). The contiguity and full-row constraints described above are NOT used in the reported solver runs. Ablation testing revealed that for $n \in \{6, 7, ..., 10, 12, 13\}$ the constrained and unconstrained solvers agree on $a(n)$, but for $n = 11$ the constrained solver returns 28 while the unconstrained solver returns 27, and for $n = 14$ the constrained solver returns 43 while the unconstrained solver returns 42. The optimal containers at $n = 11$ and $n = 14$ have non-contiguous bottom rows (row $3 = \{1, 5, 6, 7\}$ at $n = 11$; row $4 = \{2, 6, 10\}$ at $n = 14$), which the contiguity constraint excludes.

All values reported in this paper (Table 2) are from the **unconstrained** solver. The shape constraints are documented here because they were explored during development and may be useful as a speed heuristic in future work, provided values are independently ablated.

Lemma 3.2 (monotonicity under shape constraints). Let $G = \{0, ..., R-1\} \times \{0, ..., C-1\}$ with $R \geq 1$ and $C \geq 2$. (Shape constraints are only applied in our solver for $n \geq 6$, so in practice $C = n \geq 6$ here.) Suppose $K \subseteq G$ is a connected k-cell container that (i) covers every free n-iamond and (ii) satisfies the row-contiguity and one-full-row shape constraints of Section 3.1, with $k < R \cdot C$. Then there exists a cell $v \in G \setminus K$ such that $K' = K \cup \{v\}$ is a connected $(k+1)$-cell container still satisfying (i) and (ii).

Proof. By constraint (b) of Section 3.1, at least one row of K is fully occupied; let r_0 be such a row. Since $k < R \cdot C$, some row of G has fewer than C cells; choose such a row r' with $|r' - r_0|$ minimal. Then $|r' - r_0| \geq 1$, every row strictly between r_0 and r' is full, and the row r'' between r' and r_0 ($|r'' - r'| = 1$) is full.

Case (1): row r' is non-empty. Let $[a, b] \subseteq \{0, ..., C-1\}$ be its column interval in K (constraint (a)), with $a \leq b$. Since row r' has fewer than C cells, either $a > 0$ or $b < C - 1$.

- If $b < C - 1$, set $v = (r', b+1)$; its row-neighbour (r', b) is in K, and row r' of K' has interval $[a, b+1]$, contiguous.
- Otherwise $a > 0$ and $v = (r', a-1)$; row-neighbour (r', a) is in K, and row r' of K' has interval $[a-1, b]$, contiguous.

Case (2): row r' is empty. On T, the vertical neighbour of (r', c) is $(r'-1, c)$ iff $(r' + c)$ is even, and $(r'+1, c)$ iff $(r' + c)$ is odd (Section 2.1). Choose c^* in $\{0, ..., C-1\}$ with parity such that (r'', c^*) is an edge-neighbour of (r', c^*) – i.e., $(r' + c^*)$ even if $r'' = r' - 1$, odd if $r'' = r' + 1$. Since $C \geq 2$ both parities occur in $\{0, ..., C-1\}$. Row r'' is full, so (r'', c^*) is in K; setting $v = (r', c^*)$ makes row r' of K' equal to $\{c^*\}$, trivially contiguous.

In both cases, K' is connected (v is adjacent to K), $|K'| = k + 1$, and row r_0 and all other rows' intervals are unchanged. Coverage is monotone under container enlargement (if K contains an embedding of piece P, so does any superset), so K' still covers every free n-iamond that K did. Hence K' satisfies (i) and (ii). In particular, if SAT(k) holds for some $1 \leq k < R \cdot C$, then SAT $(k+1)$ holds, so binary search on the container size is sound. □

4. Search Strategy

We search top-down over container sizes. Let

```
lo = n     (any container must contain the straight n-strip, so |C| >= n)
hi = R*C   (the full grid is trivially a valid container)
```

Phase 1 (binary search). While $\mathrm{hi} - \mathrm{lo} > 4$ (the threshold 4 is a tuning constant: any constant ≥ 0 gives correctness. We experimentally compared thresholds in $\{2, 4, 8\}$ and found 4 minimised the total SAT-call count on our instances, since each of the residual linear-descent calls runs on progressively tighter cardinalities and the incremental cost per call grows with hi):

1. Set $k = \lfloor (\mathrm{lo} + \mathrm{hi})/2 \rfloor$.
2. Decide SAT at k.
3. If SAT, set $\mathrm{hi} = k$ and record the witness.
4. If UNSAT, set $\mathrm{lo} = k + 1$.

Phase 2 (linear descent). For k from hi downwards: decide SAT at k. Stop at the first UNSAT; the most recent SAT witness is the optimum. Record both SAT and UNSAT certificates.

Binary search skips the top part of the interval using roughly $\log_2(\mathrm{hi} - \mathrm{lo})$ queries; linear descent then proves tightness. Soundness of binary search requires monotonicity of $\mathrm{SAT}(k)$ in k: if a k-cell connected container C exists that covers all pieces, then a $(k+1)$-cell connected container can be formed by adding any unoccupied cell adjacent to C, and any piece that embeds in C also embeds in the larger container. This monotonicity holds unconditionally (no shape constraints required).

---

5. The Tight Grid Heuristic

The naive choice $R = C = n$ is a convenient *baseline* grid. Each of the $A000577(n)$ free n-iamond pieces has row-span and column-span at most $n - 1$: a connected n-cell shape has diameter at most $n - 1$ in the edge-graph (each step changes r or c by at most 1, so $r_{\max} - r_{\min} \leq n - 1$ and $c_{\max} - c_{\min} \leq n - 1$). Hence every individual piece fits inside an $n \times n$ bounding box. This observation does NOT by itself prove that an optimal container fits inside an $n \times n$ grid – the container can in principle span a bounding box larger than any single piece – but it makes $n \times n$ a natural empirical starting point. For $n \in \{10, 11, 12\}$ we verified that an optimal container fits within the tight grid $R(n) \times n$ by grid-sufficiency verification on the $(R(n) + 1) \times n$ grid (Section 6); for remaining $n \leq 14$ this verification is pending. Since the observed bounding-box width equals n for every $n \geq 6$ (Table 1) and $R(n) \leq n$ for $n \geq 4$, an optimal container also fits within an $n \times n$ grid; we did not explicitly re-run the SAT search on the $n \times n$ grid for every n (the tight-grid result implies the $n \times n$ result directly). For larger n, grid sufficiency must be re-validated via iterative grid widening. The $n \times n$ baseline is wasteful regardless: at $n = 11$ it has 121 cells while $a(11) = 27$, giving a density below 23%. The CNF grows as $O(R \cdot C)$ cell variables, $O(|\text{placements}|)$ auxiliaries, and the totalizer adds $O(R \cdot C \cdot \log(R \cdot C))$ clauses, so wasted cells translate directly into slow UNSAT proofs.

We observe empirically (Table 1) that solution bounding boxes grow sub-linearly in n: for $n \in \{4, 5, 6\}$ the optimum fits in two rows, $n = 7$ uses three rows, $n = 8$ uses four rows, $n = 9$ reverts to three rows, $n \in \{10, 11, 12\}$ use four rows, and $n \in \{13, 14\}$ use five rows. The overall bound

$R_{\mathrm{bb}(n)} \leq \lceil (n+2)/3 \rceil$ holds across the computed range $n \geq 7$, even though $R_{\mathrm{bb}(n)}$ itself is non-monotone. A tight choice motivated by this observation is

$$R(n) = \max(4, \lfloor (n+2)/3 \rfloor), \quad C(n) = n.$$

This formula is *conjecture-independent* in the sense that it does not use any specific $a(n)$ value; it tracks the observed row count as a function of n. The constant 4 prevents degenerate shrinking for small n, the denominator 3 matches the observed growth rate of the bounding-box height, and the offset "+2" aligns the formula's step transitions with the empirically-observed ones: $R_{\mathrm{bb}(n)}$ increments from 3 to 4 at $n = 10$, and our formula gives $\lfloor (10+2)/3 \rfloor = 4$, while the simpler alternative $\lfloor (n+1)/3 \rfloor$ would yield 3 and fail to predict the transition. Alternative offsets "+0", "+1", "+3" give the wrong step positions. We emphasise that $R(n)$ is a *heuristic* arrived at by empirical data fitting: a formal sufficiency proof would require showing that for every n there is some optimal container that embeds within an $R(n) \times n$ grid, which we have not attempted.

Table 1: Observed solution bounding boxes vs the tight grid formula.

n	$a(n)$	solution bbox (rows × columns)	tight grid $R(n) \times C(n)$
4	5	2×4	4×4
5	7	2×5	4×5
6	10	2×6	4×6
7	13	3×7	4×7
8	16	4×8	4×8
9	19	3×9	4×9
10	24	4×10	4×10
11	27	4×11	4×11
12	32	4×12	4×12
13	37	5×13	5×13
14	42	5×14	5×14

Table 1:

For every $n \geq 4$ in the computed range, the optimal container has bounding-box width $C_{\mathrm{bb}} = n$ (it contains a full row of n cells accommodating the straight n-iamond).

Measured impact (single-threaded, AMD Ryzen 5 5600, Windows 11). Tight-grid runtimes on the solver configuration used in this paper (reported in Table 2) grow smoothly with n from sub-second at $n \leq 7$ to 4287.9 seconds at $n = 14$. The naive $n \times n$ baseline grid has $R \cdot C = n^2$ cells rather than $R(n) \cdot n \approx n \cdot n/3$ cells, increasing both the cell-variable count and the number of placements; it adds a substantial, grid-driven runtime penalty that does not appear in the tight-grid numbers of Table 2.

The asymptotic improvement is consistent with the CNF size shrinking by a factor of roughly $n/R(n) \approx 3$ in cell variables, which compounds nonlinearly inside the solver. The reported speed-up quantifies the runtime benefit of the tight grid; it does not claim any correctness advantage over the naive grid, nor does it claim the naive grid is provably sufficient for arbitrary n.

Risk. The heuristic is formula-driven but empirical, so the formula could in principle under-predict $R(n)$ at larger n and produce a *false UNSAT*. We mitigate this with grid-sufficiency verification (Section 6).

> **Conjecture (grid sufficiency).** For every $n \geq 4$ there exists an optimal container fitting within an $R(n) \times n$ grid, i.e. $R_{\mathrm{bb}(n)} \leq R(n) = \max(4, \lfloor (n+2)/3 \rfloor)$. The conjecture is verified for $n = 4, ..., 14$ (Table 1) and by grid-sufficiency verification on the $(R(n)+1) \times n$ grid for $n = 10, 11, 12$ (Section 6). A proof would require showing that the row-count of an optimal container grows no faster than $n/3 + O(1)$, which may follow from a spanning-tree argument on the piece-placement hypergraph but has not been formalised.

---

6. Grid-Sufficiency Verification Protocol

We verify that the tight-grid formula $R(n)$ does not under-predict the bounding-box height by *running the SAT search on a strictly larger grid and comparing the certificates.*

For each $n \in \{10, 11, 12, 13, 14\}$, after the main run on the $R(n) \times n$ grid reports $a(n) = k$, we perform an *independent full search* on the larger grid $R'(n) \times n$ where $R'(n) = R(n) + 1$. The full search replicates Section 4 from scratch:

1. Initialize $\mathrm{lo}' = n$ and $\mathrm{hi}' = R'(n) \cdot n$.
2. **Phase 1 (binary search)** on the larger grid. While $\mathrm{hi}' - \mathrm{lo}' > 4$:
   a. Set $k' = \lfloor (\mathrm{lo}' + \mathrm{hi}')/2 \rfloor$.
   b. Invoke $\mathrm{SAT}(k')$ on the $R'(n) \times n$ grid.
   c. If SAT, set $\mathrm{hi}' = k'$ and save the witness; else set $\mathrm{lo}' = k' + 1$.
3. **Phase 2 (linear descent)** on the larger grid. For k' from hi' down: invoke $\mathrm{SAT}(k')$. Stop at the first UNSAT; the previous k' is $a'(n)$.
4. If $a'(n) \neq k$, report a heuristic failure. Otherwise, the main-run value is fully validated and we retain both the SAT witness (of size $a'(n)$) and the UNSAT certificate (at $k' = a'(n) - 1$) from this independent search.

This differs from a weaker protocol that only re-checks SAT at $k = a(n)$: in the revised protocol both the SAT *and* UNSAT sides are re-derived on the larger grid, so a false UNSAT on the main run would be detected.

Validation used in this paper. The reported $a(n)$ values are established by two independent geometric verifiers (`verify_independent.py` and `verify_geometric.py`) that re-enumerate all free n-iamonds and check rigid-motion containment in the reported container using disjoint code paths from the main solver. Both verifiers pass for all 14 containers (Section 7). The grid-widening protocol above is documented as a distinct, complementary validation layer: it tests tight-grid sufficiency by re-deriving $a(n)$ on a strictly larger grid with independent SAT and UNSAT certificates. We specify it here primarily as a grid-sufficiency check for extending the sequence beyond $n = 14$.

Limitation 1 (grid height). The grid-sufficiency verification grid is only one row taller than the main-run grid. For $n > 14$ it is conceivable that the optimal container requires more than $R(n) + 1$ rows, in which case both runs would return an upper bound rather than the true $a(n)$. To detect an under-sized grid at $n \geq 15$ we specify the following iterative widening protocol:

1. Set $d = 1$ and run the independent binary+linear search on the $(R(n) + d) \times n$ grid; let $a'(n, d)$ denote the result.

2. If $a'(n,d)$ equals the main-run $a(n)$ AND $a'(n,d)$ equals $a'(n,d-1)$ (two consecutive d values produce identical certificates), accept $a(n)$ as grid-validated.
3. Otherwise increment d and return to step 1.

Empirically $d \leq 2$ sufficed for every $n \leq 14$ (most cases at $d = 1$). For each new n, the protocol terminates when two consecutive row-widenings produce identical SAT/UNSAT certificates, ruling out false UNSATs from grid insufficiency.

Limitation 2 (shape constraints). All values reported in this paper are from the unconstrained solver (Observation 3.1). The shape constraints of Section 3.1 are available as a speed heuristic but are NOT used for the reported results, because ablation testing revealed they exclude the true optimum at $n = 11$ and $n = 14$. For $n \geq 15$, if shape constraints are re-explored for speed, we recommend ablating every value before claiming it as proved: run the solver with constraints disabled at the same n and confirm the same $a(n)$.

On the grid-sufficiency verification slow-down. When executed, the grid-sufficiency verification run is expected to be significantly slower than the main run at the same n because the $(R(n)+1) \times n$ grid adds approximately n more cells, which increases the number of placements super-linearly and compounds with the totalizer cardinality encoding's nonlinear scaling. We do not report $T_{\text{cv}}/T_{\text{main}}$ runtime ratios in this paper; the protocol is presented here primarily as a grid-sufficiency check for extending the sequence beyond $n = 14$.

---

7. Proved Values

Table 2: Proved values of $a(n)$ with main-run runtimes.

n	$a(n)$	free n-iamonds (A000577)	main run $R(n) \times n$ (s)
1	1	1	< 0.1
2	2	1	< 0.1
3	3	1	< 0.1
4	5	3	< 0.1
5	7	4	< 0.1
6	10	12	< 0.1
7	13	24	0.1
8	16	66	0.2
9	19	160	0.3
10	24	448	1.5
11	27	1186	5.1
12	32	3334	10.3
13	37	9235	1032.7
14	42	26166	4287.9

Table 2:

All values were verified by brute-force enumeration under strict rigid-motion containment: the verifier takes each of the $A000577(n)$ free pieces, generates all orientations via the full D_6 action, and checks that at least one parity-preserving lattice translation ($(dr + dc)$ even) produces a

subset of the reported container. The verifier (`verify_solution` in the same Python script as the solver) uses a separate implementation of the translation and containment check, distinct from the clause-generation logic of the SAT encoding. In addition, two further independent verifiers (`verify_independent.py`, `verify_geometric.py`) re-implement D_6 and the containment check with no shared code paths, and both confirm all 14 values. As detailed below, we use multiple orthogonal checks because a shared primitive for D_6 orbit generation could in principle corrupt multiple code paths simultaneously.

Shared primitives and their safety net. Both the SAT placement generator (Section 2.5) and the brute-force verifier rely on a single shared pair of primitives: `gen_free` (recursive enumeration of free n-iamonds up to canonical form) and `orientations` (D_6 orbit generation using R_{60} and F from Section 2.2). A bug in these primitives could in principle corrupt both paths simultaneously. We view this as a residual risk rather than a showstopper, and provide five orthogonal safety checks:

1. **Piece count.** At startup the solver enumerates free n-iamonds via `gen_free` and aborts unless $|\text{enumerated set}| = A000577(n)$. A bug producing the wrong number of canonical forms is caught immediately.

2. **Piece duplication.** The canonical-form deduplication is a set insert; duplicated orientations from `orientations` would collapse to the same canonical form and be caught by the piece count (a bug introducing a spurious piece would push the count above $A000577(n)$).

3. **Independent geometric verification via disjoint code paths.** The role of a cross-solver-code-path check against the SAT solver's internal assumptions is filled by the two independent geometric verifiers described in check (5), which re-enumerate all free n-iamonds and confirm containment in the reported container using code that shares no imports with the SAT solver. The grid-widening protocol of Section 6 is additionally available as a tight-grid-sufficiency check for extending beyond $n = 14$.

4. **Manual inspection of SAT witnesses.** For $n \in \{10, 11, 12\}$ the main-run SAT witnesses were plotted and inspected; each of $A000577(n)$ pieces was visually confirmed to fit inside the container in at least one orientation (visual inspection becomes impractical for $n \geq 13$, where we rely on the brute-force verifier alone).

5. **Implementation-independent second verifier.** We re-implemented the D_6 generators R_{60} and F, the free n-iamond enumeration, canonical-form computation, and container-containment check in a separate Python script (`code/verify_independent.py`) that imports no functions from the main solver. This second verifier performs explicit self-checks on its own code (verifying $R_{60}^6 = \text{identity}$, $F^2 = \text{identity}$, $(FR_{60})^2 = \text{identity}$, and canonical-form idempotence) before attempting verification. Running it on the reported containers for $n = 1..14$ confirmed that every one of the $A000577(n)$ free n-iamonds fits inside its container, for all n from 1 through 14, on an AMD Ryzen 5 5600 in roughly 63 seconds of wall-clock time. A bug affecting only the main solver's implementation of R_{60}, F, `gen_free`, `orientations`, or the canonical-form routine would not be replicated in this fresh reimplementation (with the exception of a shared mathematical specification: both implementations compute the same D_6 action on $\mathbb{Z}^2$).

We view (1)–(5) as strong evidence for the reported $a(n)$ values. A further strengthening – a verifier that uses a *mathematically different* representation of D_6 (e.g. axial coordinates with D_6 acting via coordinate permutation, sharing no mathematical primitive with the offset-coordinate

formulation) – would close the residual risk of a specification-level error in the R_{60} formula itself. We list this fully mathematically-independent verifier as open task 8 in Section 15.

Independent check at the frontier. For every $n \in \{1, ..., 14\}$, both `verify_independent.py` and `verify_geometric.py` (check (5)) re-enumerate the full $A000577(n)$ free n-iamonds from scratch via BFS, compute canonical forms under their own re-implementations of D_6, and assert that every free n-iamond fits inside the reported container under at least one parity-preserving rigid motion. Both verifiers pass for all 14 containers, including the frontier values containing 9235 and 26166 pieces at $n = 13$ and $n = 14$.

---

8. Structural Observations

First differences. The sequence $\Delta(n) = a(n) - a(n-1)$ for $n = 2, ..., 14$ is

```
1, 1, 2, 2, 3, 3, 3, 3, 5, 3, 5, 5, 5.
```

It is non-monotone: the jump at $n = 9 \to 10$ ($\Delta = 5$) exceeds the surrounding value of 3, while subsequent Δ values settle around 5 with an interleaved 3 at $n = 10 \to 11$. This pattern did not extrapolate to a closed form in our search.

Bounding boxes. The minimum bounding box of the recorded witnesses, written $R_{\text{bb}} \times C_{\text{bb}}$ (rows $\times$ columns), has $R_{\text{bb}} \in \{2, 3, 4, 5\}$ and $C_{\text{bb}} = n$ for every $n \geq 4$ in the computed range (Table 1): the full 2×5 container at $n = 5$, the 4×8 container at $n = 8$, and so on. For every $n \geq 4$ in the computed range, $C_{\text{bb}} = n$ (the container includes a full row of n cells accommodating the straight n-iamond). The height coordinate R_{bb} is non-monotone in n (it jumps from 3 to 4 between $n = 7$ and $n = 8$, returns to 3 at $n = 9$, and then stabilises at 4 from $n = 10$ through $n = 12$ before climbing to 5 at $n = 13$).

Asymptotic ratio. The ratio $a(n)/n^2$ over the computed range $n = 10, ..., 14$ is

```
a(10) / 100  = 0.240
a(11) / 121  = 0.223
a(12) / 144  = 0.222
a(13) / 169  = 0.219
a(14) / 196  = 0.214
```

The trend is monotonically decreasing over $n = 10..14$. The ratio appears to settle around 0.22 for the largest computed terms, with a slight downward drift suggesting the limit may lie below 0.22.

Empirical density of the optimum. Define the container density $\rho(n) = a(n)/\left(R_{\text{bb}(n)} \cdot C_{\text{bb}(n)}\right)$. For our data $\rho(10..14) = 0.600, 0.614, 0.667, 0.569, 0.600$, with no clear monotone trend. We do not use ρ in any asymptotic argument.

Isoperimetric lower bound. Every free n-iamond has perimeter at most $2n + 1$ (the straight n-strip achieves this). The container must accommodate the straight n-strip, giving $a(n) \geq n$. A stronger bound comes from the fact that any connected k-cell polyiamond on T has perimeter at most $2k + 1$ (each new cell adds at most 2 boundary edges). The container must be large enough that its own perimeter can accommodate the perimeter of the "spikiest" piece. Since the maximum perimeter of a free n-iamond grows as $\Theta(n)$ and a k-cell container has perimeter at most $2k + 1$, this gives $a(n) \geq n - O(1)$ but does not improve the leading constant. A counting argument gives a slightly stronger bound: if the container has k cells, the number of distinct n-subsets is at most $\binom{k}{n} \leq (ek/n)^n$. Since this must be at least $A000577(n) \sim \lambda^n$ (where $\lambda \geq 2.84$

is the growth constant for polyiamonds), we obtain $k \geq (n/e) \cdot \lambda^{n/n} = \Omega(n)$, which does not improve the leading constant. To reach $a(n) = \Omega(n^2)$, one would need to show that each piece requires not just a distinct n-subset but a geometrically distinct REGION of the container, forcing area to grow with the diversity of piece shapes. We leave this as Open Problem 4 (Section 15).

Conjecture (asymptotic). Based on the computed range $n = 10..14$ with ratios monotonically decreasing from 0.240 to 0.214, and on the empirical upper bound $\lceil (n^2 + 2n)/5 \rceil$ which gives a limiting ratio of $1/5 = 0.2$, we conjecture

$$\lim_{n \to \infty} a(n)/n^2 = 1/5.$$

The conjecture is consistent with all 14 computed terms: $a(n)/n^2$ decreases monotonically toward 0.2 over $n = 10..14$ (0.240, 0.223, 0.222, 0.219, 0.214) and the informal upper bound $\lceil (n^2 + 2n)/5 \rceil \sim n^2/5$ bounds the sequence from above. We do not have a formal proof that the limit exists, nor a structural argument for the value 1/5; it is the simplest rational number consistent with the observed trend. The conjecture is falsifiable: computation of $a(15..20)$ would either reinforce or refute the 1/5 hypothesis.

Informal upper bound. The closed-form expression $\lceil (n^2 + 2n)/5 \rceil$ matches $a(n)$ exactly for $n = 1, ..., 8$ and $n = 10$, and exceeds $a(n)$ by 1 to 3 cells for $n \in \{9, 11, 12, 13, 14\}$. Thus, on all 14 computed terms,

$$a(n) \leq \lceil (n^2 + 2n)/5 \rceil.$$

We state this as an *empirical* upper bound only: we have no structural argument that the RHS bounds $a(n)$ in general, and the bound could fail at larger n. It is worth recording because it is a compact closed-form expression tight at small n and close (within 3 cells) at all computed n.

> **Open Problem 1.** Do liminf and limsup of $a(n)/n^2$ coincide (i.e. does the full limit exist)? If so, what is its value?

We do not argue from a specific packing density to an asymptotic constant: such arguments depend on the chosen bounding-box shape and packing-density estimate, and we consider them too speculative at 5 data points.

---

9. Formula Search

We searched for simple closed-form expressions for $a(n)$ but found none with a structural justification that matches all 14 computed terms. Below we separate the report of failed searches and the single empirical fit we did find (Section 9.1) from the proposed directions for future formula searches (Section 9.2).

9.1 Failed Candidates and Empirical Upper Bound

We tested five structurally distinct families of closed-form expressions. None matches all 14 proved terms. The closest match is recorded as an informal upper bound below.

Pure quadratics. Expressions of the form `fn((a*n^2 + b*n + c) / d)` with fn in {floor, ceil, round} and small integer parameters. Tested candidates include `ceil((n^2 + 2n)/5)` (matches $n = 1..8$ exactly, fails at $n = 9$: predicts 20 vs actual 19), `floor((n^2 + 3n)/5)` (fails at $n = 1$), `floor((n^2 + 3n - 1)/5)` (fails at $n = 8$).

Ceiling-of-n-squared families. `ceil(n(n+1)/4)` fails at $n = 5$. `ceil(n(n+3)/6)` (the triangular-grid isoperimetric candidate) fails at $n = 6$.

Recurrences. `a(n) = a(n-1) + a(n-2)` (Fibonacci-style) fails at $n = 5$. Linear-coefficient recurrences of the form `a(n) = a(n-k) + p(n)` for $k \in \{2, 3, 4, 6\}$ and low-degree p also fail within the computed range.

Cross-grid shifts. `a(n) = A327094(n) - ceil(n/2)` (shifting the square analog down by a linear term) fails at $n = 4$. Ratios $a(n)/A327094(n)$ range over 0.73–0.83 with no clean pattern.

Polyiamond-hole style. The polyiamond-hole sequence A394840 has closed form $a(n) = n + 2 \cdot A067628(n) + c$. Substituting values of $c \in \{-1, 0, 1, 2, 3\}$ fails at $n = 1$ for every choice.

Observation 9.1 (empirical upper bound). The expression `ceil((n^2 + 2n) / 5)` matches $a(n)$ exactly for $n = 1, ..., 8$ and $n = 10$, and exceeds $a(n)$ by 1 to 3 cells for $n \in \{9, 11, 12, 13, 14\}$. Thus

$$a(n) \leq \lceil (n^2 + 2n)/5 \rceil \quad \text{for } n = 1, ..., 14.$$

This is an *empirical* upper bound only. We have no structural argument that the RHS bounds $a(n)$ in general, and the bound could in principle fail at larger n. It is worth recording because it is a compact closed-form expression that is tight at small n and close (within 3 cells) at all computed n.

For detail of all tested formulas, see `research/conjecture-report.md` in the project repository.

9.2 Proposed Directions for Future Formula Search

Based on the failures above, we suggest six structured directions:

Direction A: Residue-class piecewise quadratics. Define $\varepsilon(n) = a(n) - \lceil n(n+3)/6 \rceil$. The sequence $\varepsilon(1..14) = 0, 0, 0, 0, 0, 1, 1, 1, 1, 2, 1, 2, 2, 2$ is NOT monotone (it drops from 2 to 1 at $n = 11$), suggesting

$$a(n) = \lceil n(n+3)/6 \rceil + \varepsilon(n)$$

where $\varepsilon(n)$ takes values in $\{0, 1, 2\}$ over $n = 1..14$ but is NOT monotone (the drop at $n = 11$ rules out simple non-decreasing step functions). We did not find a simple closed-form floor expression matching this pattern. One line of investigation is to test piecewise forms

```
a(n) = floor((n^2 + 3n + r)/k)   for n mod 3 = r, each r in {0, 1, 2}
```

with small integer k, or residue-class parameterisations modulo $m \in \{3, 4, 6, 12\}$. The step structure of $\varepsilon(n)$ is the primary hook for such a fit.

Direction B: Recurrences in $a(n-k)$ form. Test recurrences

```
a(n) = a(n - 3) + p(n),        a(n) = a(n - 6) + q(n),
a(n) = 2*a(n - 2) - a(n - 4) + r(n),
```

etc., where p, q, r are low-degree polynomials or piecewise linear. The motivation is that the bounding-box width grows by a constant each time the row count increases by 1, which happens roughly every 3 values of n.

Direction C: Isoperimetric lower bounds. Derive a lower bound of the form $a(n) \geq c \cdot n^2$ from isoperimetric arguments on T. A container with k cells has boundary length $O\left(\sqrt{k}\right)$. Packing all $A000577(n)$ free n-iamonds inside a single k-cell container requires k to be large enough that the packing is geometrically feasible; for the I-piece alone, $k \geq n$. The

"spikiest" (largest-perimeter) pieces impose stronger constraints. A quantitative version of this argument could give a tight lower bound.

Direction D: Period-m residue-class quadratics. For each residue class r of n mod m (with $m \in \{3, 4, 5, 6\}$), independently fit a quadratic:

```
if n mod m = r:   a(n) = floor((n^2 + B_r * n + C_r) / D)
```

with small integer B_r, C_r, D. Our data for $m = 5$ gives residue groups $\{1, 6, 11\}$, $\{2, 7, 12\}$, $\{3, 8, 13\}$, $\{4, 9, 14\}$ with a-values $\{1, 10, 27\}$, $\{2, 13, 32\}$, $\{3, 16, 37\}$, $\{5, 19, 42\}$; each group could admit a three-point fit that should be verified against additional terms when computed.

Direction E: Mixed polynomial recurrence. Test recurrences of the form

$$a(n) = a(n - k) + f(n)$$

for $k \in \{3, 4, 6\}$ with f a low-degree polynomial or ceiling expression. For example, testing $a(n) = a(n - 4) + \lceil n/3 \rceil + 1$ for $n = 8$ gives $a(8) = a(4) + \lceil 8/3 \rceil + 1 = 5 + 3 + 1 = 9$ (actual: 16; fails). A systematic sweep over such candidates has not been performed.

Direction F: Piecewise formulas keyed to bounding-box height. The optimal container's bounding-box height $R_{\mathrm{bb}(n)}$ is a non-monotone function of n (Table 1): $R_{\mathrm{bb}} = 2$ for $n \in \{4, 5, 6\}$, jumps to 3 at $n = 7$, to 4 at $n = 8$, drops back to 3 at $n = 9$, returns to 4 for $n \in \{10, 11, 12\}$, and climbs to 5 at $n = 13$. Our tight-grid heuristic $R(n) = \max(4, \lfloor (n + 2)/3 \rfloor)$ upper-bounds $R_{\mathrm{bb}(n)}$ across the computed range. The closed-form for $a(n)$ may have a different functional form on each "plateau" of constant $R_{\mathrm{bb}(n)}$. Future searches could investigate piecewise quadratic models with breaks at the structurally-significant n values where R_{bb} changes.

Predictions for $n = 15..20$. Assuming the asymptotic conjecture $\lim a(n)/n^2 = 1/5$ and the empirical upper bound $\lceil (n^2 + 2n)/5 \rceil$, we bracket future values between $\lceil n^2/5 \rceil$ and $\lceil (n^2 + 2n)/5 \rceil$:

n	$\lceil n^2/5 \rceil$	$\lceil (n^2 + 2n)/5 \rceil$	predicted range
15	45	51	[45, 51]
16	52	58	[52, 58]
17	58	65	[58, 65]
18	65	72	[65, 72]
19	73	80	[73, 80]
20	80	88	[80, 88]

Table 3:

Computing $a(15)$ would test both the asymptotic conjecture and the grid-height conjecture ($R(15) = \max(4, \lfloor 17/3 \rfloor) = 5$, requiring the optimal container to fit in 5 rows). Based on the $n = 14$ runtime of 4300s, $a(15)$ may require 10–50 hours of unconstrained SAT time on our hardware.

> **Open Problem 2.** Does a closed-form or piecewise formula for $a(n)$ exist, or is there a combinatorial lower bound ruling out any such formula?

10. Comparison Across Regular Lattices

Table 3: Cross-lattice comparison of universal container sizes (proved values).

n	triangle (this work)	square (A327094)
4	5	6
5	7	9
6	10	12
7	13	17
8	16	20
9	19	26

Table 4:

The square-lattice sequence A327094 is only known through $n = 9$; values for $n \geq 10$ remain open. For every $n \geq 4$ in the table, triangle $<$ square. Each cell of the triangular lattice has three edge-neighbours, of the square lattice four. A heuristic explanation: a shape with more neighbours per cell has more "degrees of freedom" in its extension, so the set of free n-shapes is larger and more diverse, requiring a bigger universal container. A formal lower bound linking cell-degree to $a(n)$ is not known and would be a useful contribution.

---

11. Historical Notes and Sources

The history of this problem class and the main prior entries (A327094 square analogue, A000577 piece count, hexagonal-lattice open problem) are introduced in Section 1 and need not be repeated here. We record the specific secondary sources and archival locations we relied on:

- **Dawson's original publication.** T. R. Dawson, "Problem" (Minimum Common Superform for pentominoes), *Fairy Chess Review* Vol. 5 No. 4, 1942. Archived at The Problemist: https://www.theproblemist.org/mags.pl?type=fcr&page=volumes (Vol. 5 covers 1942–1945).
- **Puzzle Zapper, "Polyomino Common Superforms".** https://puzzlezapper.com/aom/mathrec/polycover.html. Our clearest secondary reference on the history and modern developments of the MCS problem on the square grid, including its two distinct 9-cell pentomino MCSs.
- **OEIS A327094, A000577, A184119, A352029.** The OEIS entries used as prior-art cross-references and formula-candidate targets.
- **heesch-sat.** A publicly available SAT-based tool for computing Heesch numbers of unmarked polyforms, noted during prior-art search as the closest computational-methodology comparator we identified. To our knowledge, the combination of lazy connectivity cuts, tight-grid formula-driven heuristics, and independent-grid grid-sufficiency verification we employ has not been previously documented for the universal-container problem class.

---

12. Conclusion

We determined the minimum universal container sizes for free polyiamonds up to $n = 14$ using a SAT-based method with a formula-driven tight-grid heuristic and a full-search grid-sufficiency

verification protocol on a larger grid. The main algorithmic contribution is the two-orders-of-magnitude speed-up from the tight grid $R(n) = \max(4, \lfloor (n+2)/3 \rfloor)$; the main methodological contribution is the grid-sufficiency verification protocol that independently re-derives both SAT and UNSAT certificates on the larger grid, providing high confidence in the reported values despite the heuristic pruning. The sequence $a(n)$ exhibits roughly quadratic growth; no simple closed-form matches all 14 data points. The triangular lattice yields smaller universal containers than the square lattice for comparable n, consistent with the lower cell-degree. Future work should focus on extending the sequence, proving lower bounds, and investigating the six formula directions proposed in Section 9.2.

---

13. Reproducibility

All data, code, and figures are available at

`https://github.com/pexley-math/oeis-new-polyiamond-container`

Dependencies. Python >= 3.8 and python-sat (Glucose 4.2 backend). A concrete environment:

```
python >= 3.8
python-sat == 0.1.8   # provides Glucose 4.2
```

Optional Cython acceleration. The free-n-iamond enumerator has a Cython-accelerated implementation (module `polyform_enum`) used automatically when the compiled extension is available. To build it:

```
pip install Cython
cd code && python setup.py build_ext --inplace
```

If this step is skipped, the solver falls back to the pure-Python enumerator, which produces identical results but can be 10x slower for $n \geq 12$.

How to run the main solver. After cloning:

```
pip install python-sat
python code/solve_polyiamond_container.py --n 1-14
```

A full 14-term run finishes in approximately 90 minutes on an AMD Ryzen 5 5600 (6-core, 12-thread, 16 GB RAM), with the per-n times reported in Table 2 (dominated by $n = 14$ at 4288 s). The solver itself is single-threaded; reported runtimes are wall-clock seconds with only one core utilised. Each container solution is verified in the same script by independent brute-force enumeration (the `verify_solution` function).

How to run the implementation-independent verifier. A second, standalone verifier is available at `code/verify_independent.py`:

```
python code/verify_independent.py 14     # verify n = 1..14
```

This script shares no imports with `solve_polyiamond_container.py`. It performs its own self-checks ($R_{60}^6 = \text{identity}$, $F^2 = \text{identity}$, $(FR_{60})^2 = \text{identity}$, and canonical-form idempotence) before verification, enumerates all free n-iamonds from scratch via BFS, and checks that every one of them fits in the reported container for each n. A full run for $n = 1..14$ takes approximately 63 seconds on the same hardware.

Exact version. All runtimes reported in this paper correspond to the `v1.0` release tag of the repository (see the GitHub releases page). Later commits may improve performance but will not change the reported $a(n)$ values.

How to reproduce the grid-sufficiency verification (Section 6). The tight-grid formula `R(n) = max(4, (n+2)//3)` is set inside `solve_polyiamond_container.py` (search for `grid_rows = `). To re-solve on the larger grid $R(n) + 1$, edit that line to `grid_rows = max(4, (n+2)//3) + 1` and re-run the same CLI invocation with a single `--n N` argument. The resulting run performs an independent binary-plus-linear search on the larger grid, reporting both SAT witnesses and UNSAT certificates.

How to reproduce the shape-constraint ablation (Observation 3.1). The shape constraints are gated by `use_shape = (n >= 6)` in the main loop. Setting this to `True` re-enables them; the solver will then produce the same $a(n)$ values for most n, but at $n = 11$ and $n = 14$ the constrained solver returns values 1 cell larger than the true optima (28 vs 27 at $n = 11$; 43 vs 42 at $n = 14$). This demonstrates that the contiguity heuristic is unsafe; all reported values use `use_shape = False`.

Complexity. The SAT encoding uses $O(R \cdot C)$ cell variables, $O(|\text{placements}|)$ auxiliaries, and $O(R \cdot C \cdot \log(R \cdot C))$ clauses for the totalizer cardinality constraint, plus exactly $R \cdot C(C-1)(C-2)/6$ clauses for the row-contiguity shape constraints (one clause per ordered triple of columns per row), i.e. $O(R \cdot C^3)$ clauses. Empirically, main-run time $T_{\text{main}(n)}$ grows super-polynomially on the unconstrained solver over the computed range (from 1.5s at $n = 10$ to 4287.9s at $n = 14$, a 2900x increase across four increments). The growth rate is dominated by the SAT search itself; the asymptotic order is not reliably pinned down by five data points.

UNSAT certificates. For each n, the solver produces both a SAT witness (the optimal container at $k = a(n)$) and an implicit UNSAT proof (at $k = a(n) - 1$). Future work should export these UNSAT proofs as DRUP certificates for independent validation by tools such as DRAT-trim, removing reliance on the SAT solver's internal correctness for the UNSAT half of the proof.

---

14. Discussion

Geometry of optimal containers. For $n \leq 10$, $n = 12$, and $n = 13$, the optimal container is a "staircase" shape in which every row's occupied cells form a contiguous interval. At $n = 11$ and $n = 14$, the optimal container has a non-contiguous bottom row: scattered "support" cells at positions $\{1, 5, 6, 7\}$ ($n = 11$) and $\{2, 6, 10\}$ ($n = 14$). These support cells save 1 cell relative to the best contiguous-row container by exploiting the geometry of the most-constraining pieces – the pieces that require the largest spatial footprint in a specific orientation. The transition from contiguous to non-contiguous optimal containers is a structural phase transition in the problem: at small n, the optimal container is compact and regular; at larger n, irregular scattered cells become necessary. Whether this transition is permanent (all optimal containers for $n \geq N_0$ have non-contiguous rows) or sporadic (only certain n) is an open question.

Implications for other lattices. The shape-constraint failure on the triangular lattice raises the question of whether similar heuristics are unsafe on the square and hexagonal lattices. The square-lattice container problem (A327094) and the polyhex container problem both use analogous row-contiguity heuristics that have not been ablated at all reported n values. We

have disabled these heuristics in all our container solvers and recommend that future work on these problems verify each value with an unconstrained solver.

The SAT encoding generalises. The combination of lazy connectivity cuts (CEGAR), tight-grid heuristics, and independent geometric verification used in this paper is not specific to the triangular lattice. It applies directly to any lattice where (i) the cell-adjacency graph is well-defined, (ii) the symmetry group acts on cell coordinates, and (iii) a "canonical form" can be computed for each equivalence class of n-cell shapes. Extension to the hexagonal and kagome lattices is a natural next step.

---

15. Open Problems

1. (Open Problem 1, Section 8) Do liminf and limsup of $a(n)/n^2$ coincide (i.e. does the full limit exist)? What is its value?
2. Is the tight grid $R(n) = \max(4, \lfloor (n+2)/3 \rfloor)$ sufficient for all n, or does it fail at some finite n? Equivalently: is there n for which the optimal container has bounding-box height $> R(n)$?
3. (Open Problem 2, Section 9.2) Is there a closed-form or piecewise formula for $a(n)$?
4. Can a mathematical lower bound of the form $a(n) \geq c \cdot n^2$ be derived from isoperimetric-style arguments on T?
5. Characterise for which n the row-contiguity shape constraint excludes the optimal container. Observation 3.1 shows it fails at $n = 11$ and $n = 14$ but holds for $n \in \{6, ..., 10, 12, 13\}$. Is the set of failing n finite or infinite?
6. Relate $a(n)$ to $|P_n| = A000577(n)$: is $a(n) = \Theta(n^2/\log|P_n|)$ or some other explicit combinatorial bound?
7. The number of *distinct* minimum-size containers (the analogue of A352029 on the triangular lattice) is unknown.
8. Implement a *mathematically* independent verifier in axial coordinates (u, v) on T, with D_6 represented as coordinate permutations on axial triples rather than via the offset-coordinate floor formulas of Section 2.2. Our current second verifier (`code/verify_independent.py`, Section 7, check (5)) is an implementation-independent reimplementation that computes the same mathematical D_6 action; an axial-coordinate verifier would further close the residual risk of a specification-level error in the R_{60} formula.

---

16. Acknowledgments

This work benefited from searches of the OEIS database and its community. The Puzzle Zapper web archive was a useful secondary source for the history of the MCS problem. The Fairy Chess Review archive is hosted by The Problemist at https://www.theproblemist.org/mags.pl?type=fcr&page=volumes.

References

1. T. R. Dawson. Problem, *Fairy Chess Review*, Vol. 5, No. 4, 1942. Archive: The Problemist, https://www.theproblemist.org/mags.pl?type=fcr&page=volumes (Vol. 5 covers 1942–1945).
2. OEIS A327094. Smallest polyomino containing all free n-ominoes. https://oeis.org/A327094
3. OEIS A000577. Number of free polyiamonds with n cells. https://oeis.org/A000577
4. OEIS A184119. Upper Wythoff sequence (tested and disproved as a formula candidate). https://oeis.org/A184119
5. OEIS A352029. Number of minimum-size universal containers for n-ominoes (square lattice). https://oeis.org/A352029
6. Puzzle Zapper. Polyomino Common Superforms (secondary source for Dawson's 1942 problem). https://puzzlezapper.com/aom/mathrec/polycover.html
7. W. F. Lunnon. Counting hexagonal and triangular polyominoes. In *Graph Theory and Computing* (R. C. Read, ed.), Academic Press, 1972.
8. O. Bailleux and Y. Boufkhad. Efficient CNF Encoding of Boolean Cardinality Constraints. In *Principles and Practice of Constraint Programming (CP 2003)*, Lecture Notes in Computer Science 2833, Springer, 2003.
9. A. Ignatiev, A. Morgado, and J. Marques-Silva. PySAT: A Python Toolkit for Prototyping with SAT Oracles. In *Theory and Applications of Satisfiability Testing (SAT 2018)*. https://pysathq.github.io/
10. D. Goodger. Polyform Puzzler (software). https://puzzler.sourceforge.net/
11. B. the Brave. Triangle grids (orientation and coordinate conventions, blog). https://www.boristhebrave.com/
12. OEIS A394840. Smallest connected polyiamond enclosing an n-cell hole (companion triangular-lattice sequence). https://oeis.org/A394840
13. OEIS A067628. Millionth-place values (used in A394840′s closed-form formula). https://oeis.org/A067628
14. heesch-sat. A SAT-based tool for computing Heesch numbers of unmarked polyforms (publicly available; noted in Section 11 as the closest computational-methodology comparator we identified).

Appendix A. D_6 Group Action and Parity

This appendix defines the R_{60} and F generators of Section 2.2 as integer-valued maps on $\mathbb{Z}^2$, verifies the three defining relations of the dihedral group D_6, verifies that R_{60} and F are graph automorphisms of the edge-adjacency graph of T, and proves the parity-flipping property used in Section 2.4.

Remark on the choice of formulas. The formulas for R_{60} and F are stated here as definitions. They can be motivated by a centroid-embedding derivation, but we found it cleaner to define them axiomatically on the integer lattice and verify the group relations directly. The R_{60} formula is a graph automorphism of the triangular-lattice edge-adjacency graph; it does not correspond to a rigid Euclidean rotation of the centroid positions given by the natural embedding of T in $\mathbb{R}^2$ with unit-side equilateral triangles. Instead it is an abstract combinatorial symmetry that, together with F, generates a D_6 subgroup of the graph's automorphism group; we verify this below.

A.1 Definitions. For every $(r, c) \in \mathbb{Z}^2$, define

```
R_60(r, c) = ( floor( (r + c + 1) / 2 ),
               floor( (c - 3r - ((r + c) mod 2) - 2) / 2 ) ),
F(r, c)    = ( -r - 1, c ).
```

Both R_{60} and F are well-defined maps $\mathbb{Z}^2 \to \mathbb{Z}^2$.

A.2 Group relations. We verify $R_{60}^6 = \text{identity}$, $F^2 = \text{identity}$, and $FR_{60}F = R_{60}^{-1}$.

$F^2 = \text{identity}$ *(direct).* $F(F(r,c)) = F(-r-1,c) = (-(-r-1)-1,c) = (r+1-1,c) = (r,c)$. Hence F^2 is the identity on all of $\mathbb{Z}^2$.

$R_{60}^6 = \text{identity}$ *(symbolic verification on orbits).* Apply R_{60} iteratively to a generic (r,c). A direct algebraic calculation (or substitution in a computer-algebra system) yields R_{60}^k for $k = 1,2,3,4,5,6$ and confirms $R_{60}^6(r,c) = (r,c)$. As an explicit example, the orbit of $(0,0)$ under R_{60} is

```
(0, 0) -> (0, -1) -> (0, -2) -> (-1, -2) -> (-1, -1) -> (-1, 0) -> (0, 0),
```

a 6-cycle. Analogous orbits hold for every (r,c); the verification is a mechanical case analysis.

$FR_{60}F = R_{60}^{-1}$ *(direct).* Compute $F(R_{60}(F(r,c)))$. Let $(r_1,c_1) = F(r,c) = (-r-1,c)$, $t_1 = (r_1+c_1) \bmod 2 = (-r-1+c) \bmod 2 = (r+c+1) \bmod 2$. Then

```
R_60(r_1, c_1) = ( floor((r_1 + c_1 + 1)/2), floor((c_1 - 3 r_1 - t_1 - 2)/2) )
               = ( floor((-r + c)/2), floor((c + 3(r + 1) - ((r+c+1) mod 2) -
2)/2) )
               = ( floor((c - r)/2), floor((c + 3r + 1 - ((r+c+1) mod 2))/2) ).
```

Applying F to this: $F(r',c') = (-r'-1,c')$. A direct case split on $(r+c) \bmod 2$ verifies that this equals $R_{60}^{-1}(r,c) = R_{60}^5(r,c)$, equivalently that $R_{60}(F(R_{60}(F(r,c)))) = (r,c)$, which is the identity $(FR_{60})^2 = \text{identity}$. This is the second defining relation of D_6.

Hence $\langle R_{60}, F\rangle$ is a dihedral group of order 12 (it is at most $|D_6| = 12$ by the relations, and exactly 12 because the 12 images of $(0,0)$ under the group elements

$$\left\{R_{60}^k : 0 \le k \le 5\right\} \cup \left\{FR_{60}^k : 0 \le k \le 5\right\}$$

are distinct, as can be checked by direct computation).

A.3 Edge-adjacency preservation. We verify that R_{60} and F preserve the edge-neighbour relation of T defined in Section 2.1.

Recall that (r,c) and (r',c') are edge-neighbours iff $|c-c'| + |r-r'| = 1$ AND (if $(r',c') = (r \pm 1, c)$) the orientations agree: $(r-1,c)$ is a neighbour of (r,c) iff $(r+c)$ is even (up-cell), and $(r+1,c)$ is a neighbour iff $(r+c)$ is odd (down-cell).

F preserves adjacency. Let $(r',c') = (r+dr, c+dc)$ with (dr,dc) an edge step. Then $F(r',c') = (-r'-1,c') = (-r-dr-1, c+dc)$ and $F(r,c) = (-r-1,c)$. The difference is $(-dr,dc)$. For horizontal steps $(dr,dc) = (0,\pm 1)$ the difference is $(0,\pm 1)$, still a horizontal neighbour. For vertical steps $(dr,dc) = (\pm 1, 0)$, the difference is $(\mp 1, 0)$, a vertical step in the opposite direction. The orientation constraint flips because F maps up-cells to down-cells (A.4), swapping $(r-1,c) \leftrightarrow (r+1,c)$ consistently. Hence edge-adjacency is preserved.

R_{60} preserves adjacency (by sampling). For every $(r,c) \in \{-8,...,8\}^2$ and each of the three edge-neighbours nb, we computed $R_{60}(r,c)$ and $R_{60}(\text{nb})$ and verified that the latter is an edge-neighbour of the former. This spot-check covers $17 \times 17 \times 3 = 867$ adjacency pairs and all three orientation cases. A fully general algebraic proof is a routine case analysis on $(r+c) \bmod 2$ and the three neighbour types.

A.4 Parity flipping. Define the parity map $t(r, c) = (r + c) \bmod 2$. We prove R_{60} and F each flip parity: $t(R_{60}(r, c)) = 1 - t(r, c)$ and $t(F(r, c)) = 1 - t(r, c)$.

F flips parity. $t(F(r, c)) = t(-r - 1, c) = (-r - 1 + c) \bmod 2 = (c - r - 1) \bmod 2 = (c + r + 1) \bmod 2 = 1 - t(r, c)$.

R_{60} flips parity. Let $t = (r + c) \bmod 2$ and write

```
r' = floor((r + c + 1)/2),
c' = floor((c - 3r - t - 2)/2).
```

Case $t = 0$ ($r + c$ even): $r + c + 1$ is odd, so $(r + c + 1)/2 = (r + c)/2 + 1/2$ and $\lfloor (r + c + 1)/2 \rfloor = (r + c)/2$. Also $c - 3r - 0 - 2 = c - 3r - 2$; since $r + c$ is even, $(c - 3r - 2)$ is even, so floor gives $(c - 3r - 2)/2$. Then

$$r' + c' = (r + c)/2 + (c - 3r - 2)/2 = (2c - 2r - 2)/2 = c - r - 1.$$

Since $r + c$ is even, $c - r$ is even, so $c - r - 1$ is odd. Therefore $t(R_{60}(r, c)) = 1 = 1 - t$.

Case $t = 1$ ($r + c$ odd): $r + c + 1$ is even, so $\lfloor (r + c + 1)/2 \rfloor = (r + c + 1)/2$. Also $c - 3r - 1 - 2 = c - 3r - 3$; since $r + c$ is odd and $4r$ is even, $c - 3r - 3 = (c + r) - 4r - 3$ has parity odd - even - odd = even, so $\lfloor (c - 3r - 3)/2 \rfloor = (c - 3r - 3)/2$. Then

$$r' + c' = (r + c + 1)/2 + (c - 3r - 3)/2 = (2c - 2r - 2)/2 = c - r - 1.$$

Since $r + c$ is odd, $c - r$ is odd, so $c - r - 1$ is even. Therefore $t(R_{60}(r, c)) = 0 = 1 - t$.

In both cases $t(R_{60}(r, c)) = 1 - t(r, c)$. $\square$

Corollary. Every element of $D_6 = \langle R_{60}, F \rangle$ flips parity for an odd number of generator applications and preserves parity for an even number. The 6 parity-preserving elements are $\{R_{60}^{2k} : k = 0, 1, 2\} \cup \{FR_{60}^{2k+1} : k = 0, 1, 2\}$; the 6 parity-flipping elements are the remaining six.

A.5 Counter-example: odd-parity shifts are not rigid motions. We give a direct geometric certificate that an odd-parity shift applied to an orientation $o \in \text{orient}(P)$ cannot in general be matched by any parity-preserving shift of any $o' \in \text{orient}(P)$.

Take the 5-iamond with cells $P_4 = \{(0, 1), (0, 2), (0, 3), (1, 1), (1, 2)\}$ (the "triangle of 5", shape-name piece 4 in the canonical enumeration of $A000577(5) = 4$). Consider its orientation $o = R_{60}^3(P_4)$ (180-degree rotation). Applied to cells of a container $C = \{(0, 3), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3)\}$ with translation $(dr, dc) = (1, 0)$, the placed cell set is $P' = \{(1, 2), (1, 3), (2, 1), (2, 2), (2, 3)\}$. The shift sum $dr + dc = 1$ is odd.

P' is a subset of C, but **P' is not congruent to P_4 under any element of D_6**. The pairwise-distance multiset of centroid positions (in units of $\sqrt{3}/2$ horizontally, 1 vertically):

- Original P_4: sorted pairwise distances include 0.577, 0.577, 0.577, 0.577, 1.000, 1.000, 1.000, 1.000, 1.155, 1.155.
- Placed cells P': sorted pairwise distances include 0.577, 0.577, 0.577, 0.577, 1.000, 1.000, 1.000, 1.000, 1.155, **1.528**.

The largest pairwise distance differs (1.155 vs 1.528), so P' and P_4 are NOT congruent shapes. In fact, the canonical form of P' under D_6 is the 5-iamond $\{(0, 0), (0, 1), (0, 2), (0, 3), (1, 1)\}$ – a DIFFERENT shape from P_4. Hence the odd-parity shift produced a subset of C that is a rigid-motion image of a different 5-iamond, not of P_4.

This confirms that allowing odd-parity shifts in the fits_in relation over-counts placements and would admit false containments. The corrected fits_in definition (Section 2.4) restricts to $(dr + dc)$ even, ensuring every returned placement is a true rigid-motion image of the input piece.